Proiect: Procesarea de Text cu Algoritmi Paraleli

Numărarea frecvenței cuvintelor dintr-un fișier text mare

1. Topicul

Proiectul constă în numărarea frecvenței cuvintelor dintr-un fișier text de dimensiuni mari, folosind procesare paralelă. Fișierul este împărțit în segmente egale, fiecare segment este procesat pe un fir de execuție separat, iar la final rezultatele sunt combinate într-un dicționar comun.

2. Limbajul de programare utilizat

Java (JDK 17)

A fost ales datorită suportului nativ pentru fire de execuție prin clasa java.lang.Thread, disponibilă în orice instalare standard de JDK, fără dependințe externe.

3. Sisteme și framework-uri folosite

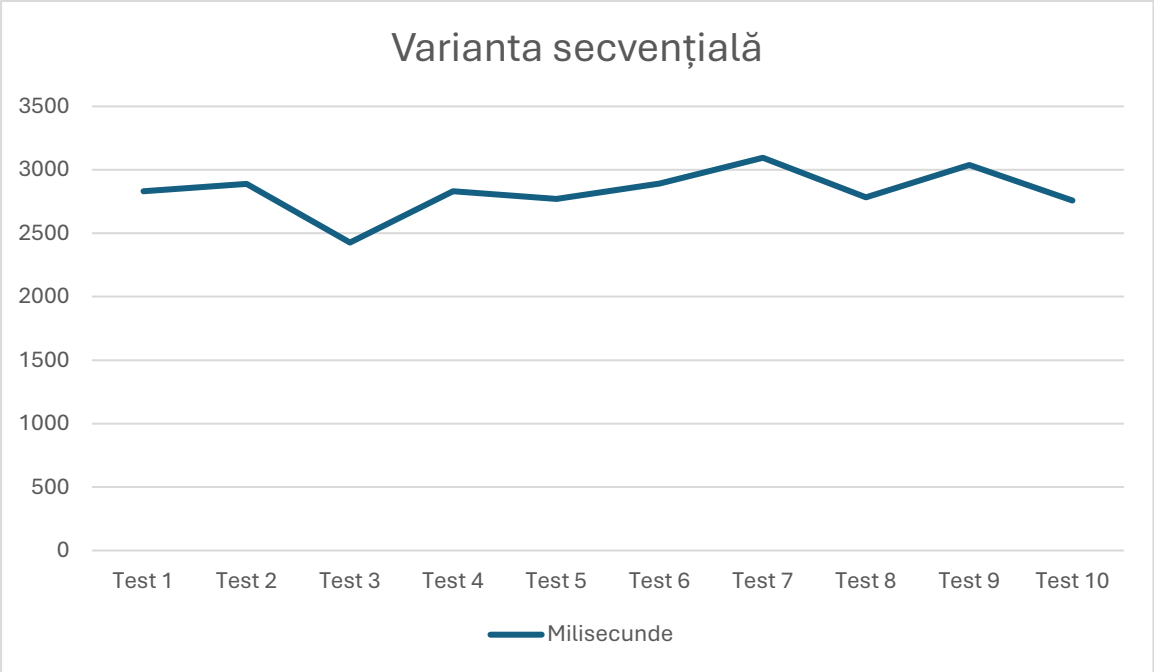
Toate componentele utilizate fac parte din biblioteca standard Java și nu necesită instalare suplimentară.

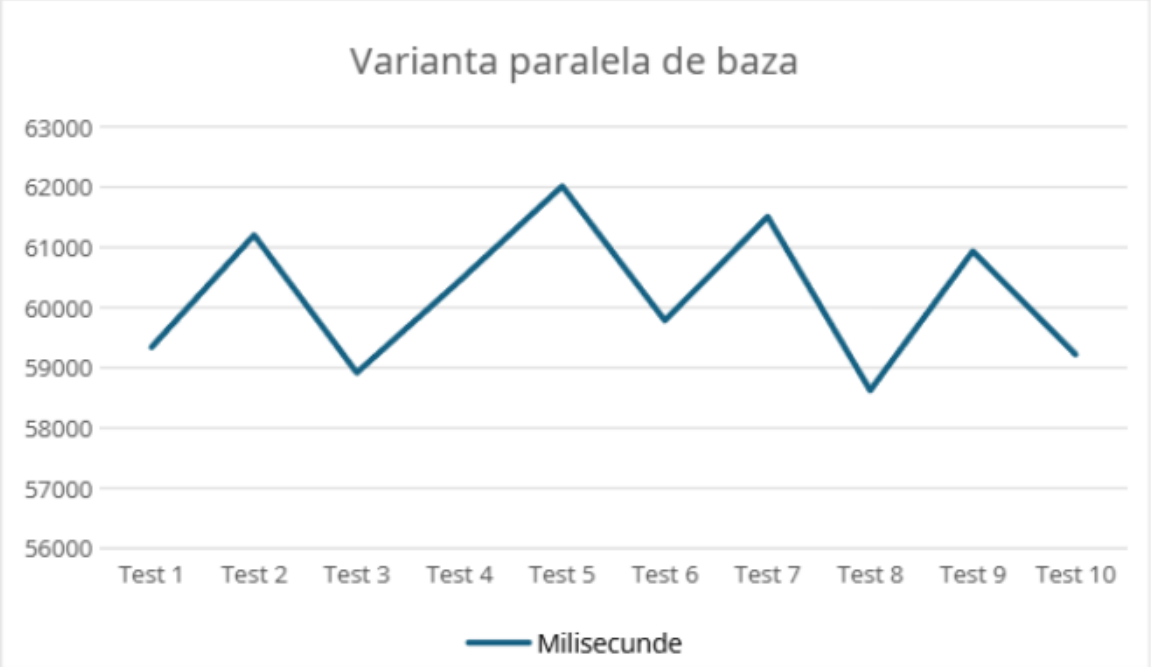
Componentă	Utilizare
java.lang.Thread	Crearea și pornirea firelor de execuție. Fiecare fir primește un segment din fișier și îl procesează independent.
HashMap<String, Integer>	Stocarea frecvenței cuvintelor pentru fiecare segment procesat de un fir.
synchronized	Prevenirea conflictelor de scriere când mai multe fire actualizează simultan dicționarul comun de rezultate.
Windows 10/11	Mediul de rulare al aplicației. JDK-ul rulează nativ pe Windows fără configurări suplimentare.

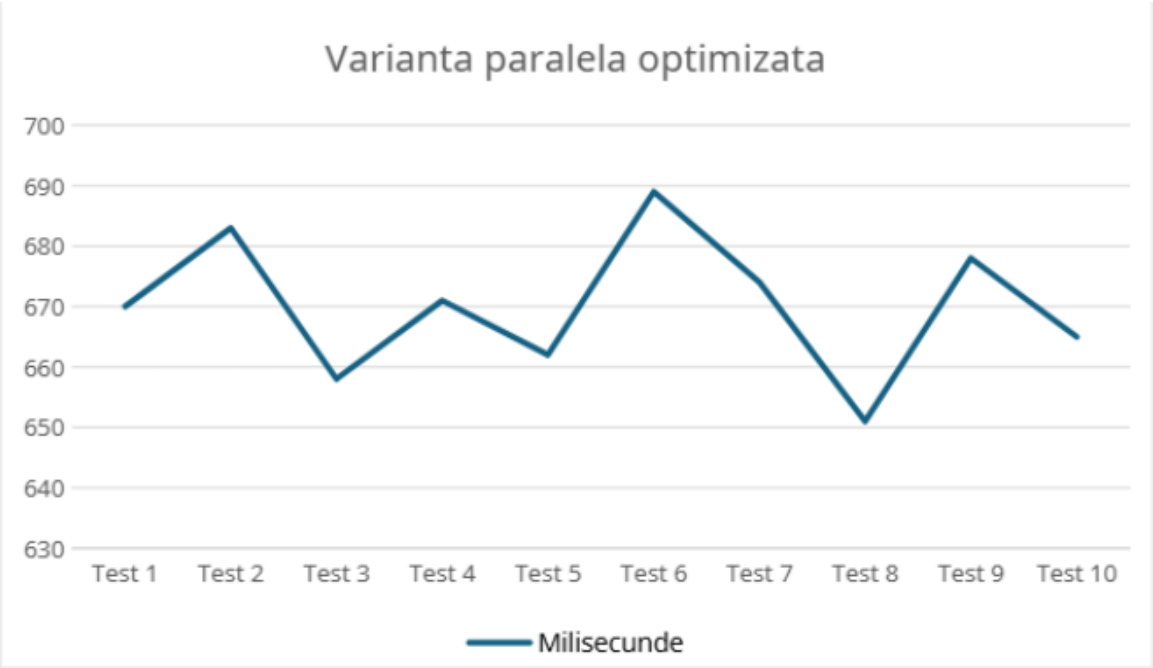
4. Informații despre mașina pe care s-a rulat codul

Procesor: AMD Ryzen 5 6600H
Memorie RAM: 16 GB
Stocare: SSD NVMe 512GB

5. Rezultate experimentale







6. Analiza si explicatii

6.1. Varianta Secventiala

Programul secvential (SecventialWordCounter) foloseste un singur fir de executie si un BufferedReader pentru a citi fisierul linie cu linie. Este varianta cea mai simpla si cea mai previzibila: I/O-ul este secvential, CPU-ul proceseaza o singura linie la un moment dat, si nu exista overhead de sincronizare sau de creare de thread-uri.

6.2. Varianta Paralela de Baza

Aceasta varianta (ParalelWordCounter) imparte fisierul in 12 segmente egale si lanseaza cate un thread pentru fiecare segment. Fiecare thread deschide fisierul independent prin RandomAccessFile si sare direct la pozitia sa din fisier folosind seek(). Problema fundamentala a acestei abordari este ca toate cele 12 thread-uri acceseaza simultan acelasi fisier de pe disc. Chiar si pe un SSD NVMe, accesele concurente ne-secventiale genereaza o contentie puternica la nivelul controller-ului de stocare, iar sistemul de operare este obligat sa arbitreze cererile concurente. Rezultatul a fost un timp de circa 15 ori mai lent decat varianta secventiala – un exemplu clar de paralelizare care agraveaza situatia.

Un alt aspect important este ca thread-ul care nu se afla la inceputul fisierului ignora prima linie citita, pentru a evita procesarea unui cuvant taiat la granita dintre segmente. Aceasta logica este corecta, dar adauga un mic overhead suplimentar.

6.3. Varianta Paralela Optimizata

Aceasta varianta (ParalelWordCounterOptimized) rezolva problema din varianta anterioara prin separarea clara a doua etape: mai intai citeste tot fisierul secvential intr-o lista din memorie folosind un singur BufferedReader, apoi distribuie aceasta lista catre cele 12 thread-uri care o proceseaza exclusiv din RAM, fara sa mai atinga discul.

Eliminand I/O-ul din zona paralela, CPU-ul poate fi folosit la capacitate maxima. Fiecare thread lucreaza independent pe segmentul sau din lista, fara nicio contention de resurse. La final, fiecare thread isi scrie rezultatele locale in dictonarul comun folosind un bloc synchronized, care blocheaza dictonarul doar pentru transferul final, nu pe tot parcursul procesarii. Aceasta strategie a condus la un timp de aproximativ 5.7 ori mai rapid decat varianta secventiala.

7. Concluzii

Proiectul a demonstrat ca paralelizarea nu aduce automat un beneficiu de performanta. Rezultatele arata clar ca bottleneck-ul cel mai important nu este procesorul, ci accesul la date.

Prima varianta paralela (ParalelWordCounter) a folosit RandomAccessFile, care obliga fiecare thread sa deschida fisierul si sa citeasca direct de pe disc. Deoarece toate cele 12 thread-uri acceseaza acelasi fisier simultan, accesele de I/O intra in competitie si sistemul de operare nu poate servi toti cititorii eficient.

A doua varianta paralela (ParalelWordCounterOptimized) a rezolvat aceasta problema prin separarea etapelor: intai citeste tot fisierul secvential intr-o lista din memorie, apoi distribuie aceasta lista catre cele 12 thread-uri care lucreaza exclusiv din RAM. Eliminand I/O-ul din zona paralela, CPU-ul a fost folosit la capacitate maxima.

Lectia principala a acestui proiect este ca pentru a obtine un speedup real din paralelizare, trebuie identificat si izolat corect bottleneck-ul. Daca procesarea este legata de I/O, mai multe thread-uri nu vor ajuta si pot chiar inrautati situatia. Paralelizarea functioneaza eficient doar atunci cand workload-ul este legat de CPU si datele sunt deja disponibile in memorie.